

Topic 5 神经网络（Neural Network）

Part 2：多层神经网络训练

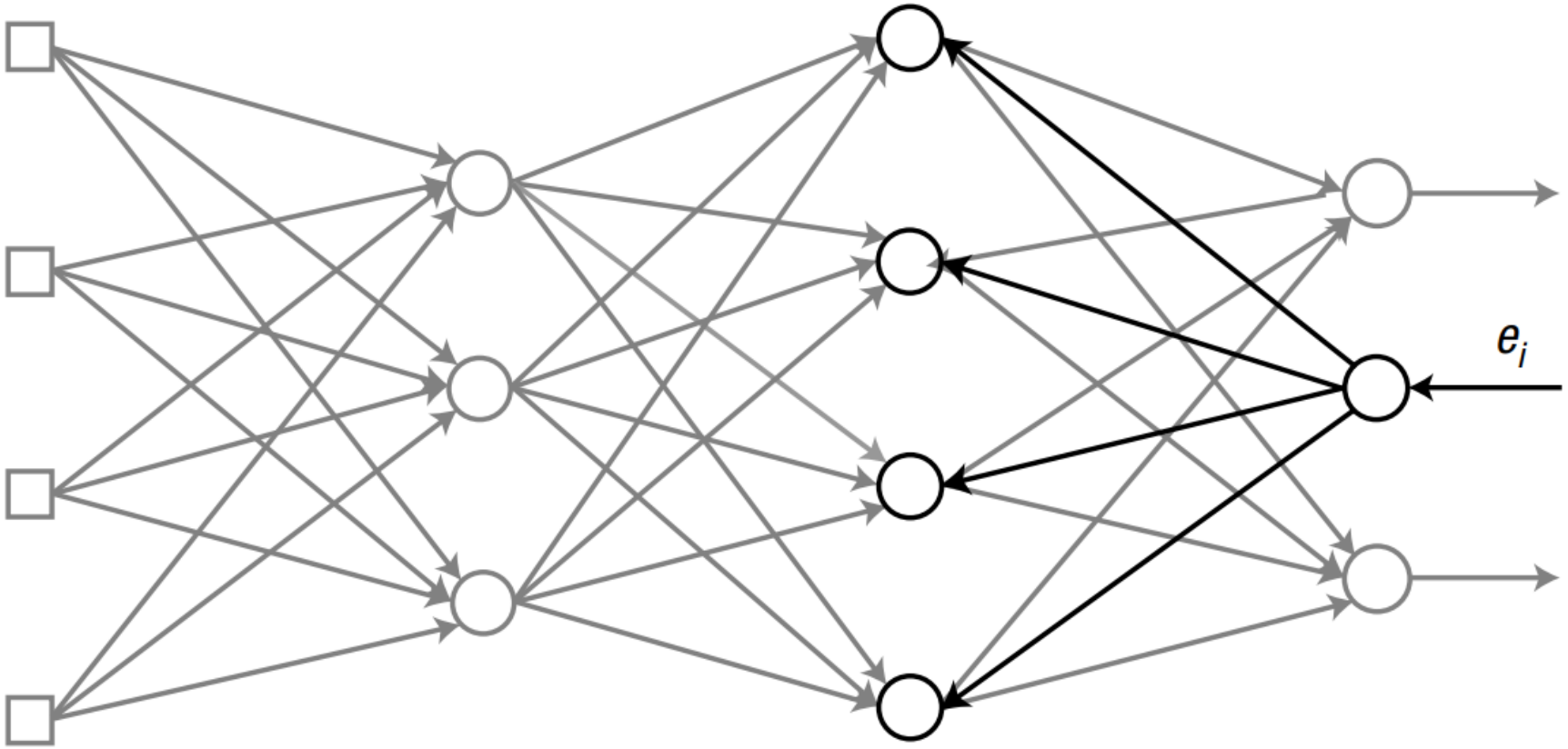
魏准
浙江大学，信息与电子工程学院
2026

总体内容

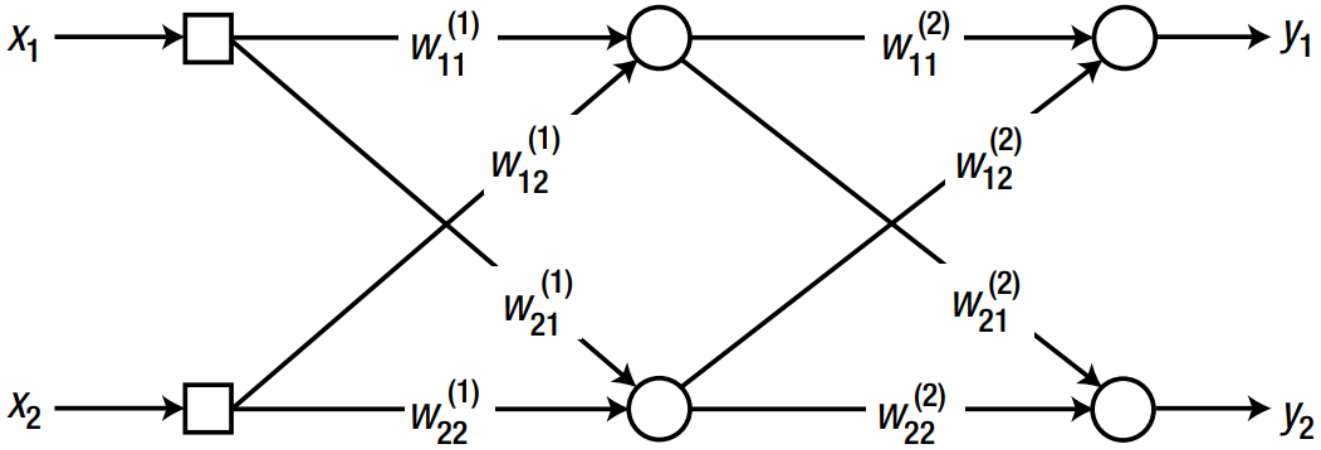
主要知识点

- 后向传输（back-propagation）
- 实例BP训练
- BP训练NN步骤
- 实验5-2
- 学习率、损失函数、 momentum
- 实验5-3

1.1：后向传输（back-propagation）



后向传输简图

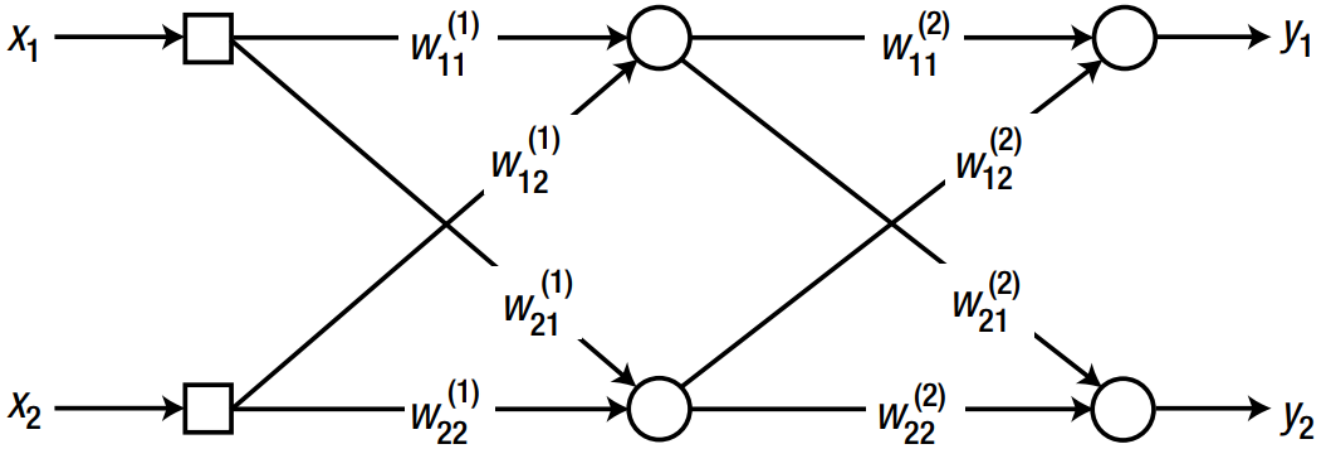


以含一层隐藏层的神经网络为例，第一层输出：

$$\begin{bmatrix} v_1^{(1)} \\ v_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(1)} & w_{12}^{(1)} \\ w_{21}^{(1)} & w_{22}^{(1)} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}$$

$$\triangleq W_1 x$$

$$\begin{bmatrix} y_1^{(1)} \\ y_2^{(1)} \end{bmatrix} = \begin{bmatrix} \varphi\left(v_1^{(1)}\right) \\ \varphi\left(v_2^{(1)}\right) \end{bmatrix}$$

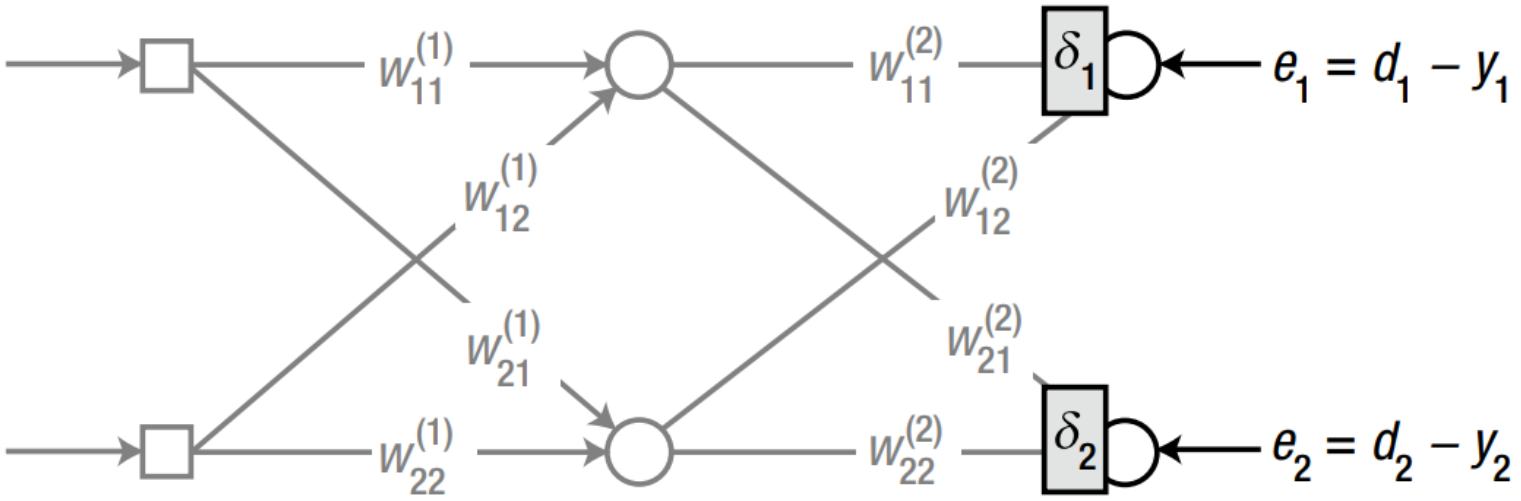


第二层:

$$\begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \begin{bmatrix} w_{11}^{(2)} & w_{12}^{(2)} \\ w_{21}^{(2)} & w_{22}^{(2)} \end{bmatrix} \begin{bmatrix} y_1^{(1)} \\ y_2^{(1)} \end{bmatrix}$$

$$\triangleq W_2 \, y^{(1)}$$

$$\begin{bmatrix} y_1 \\ y_2 \end{bmatrix} = \begin{bmatrix} \varphi(v_1) \\ \varphi(v_2) \end{bmatrix}$$



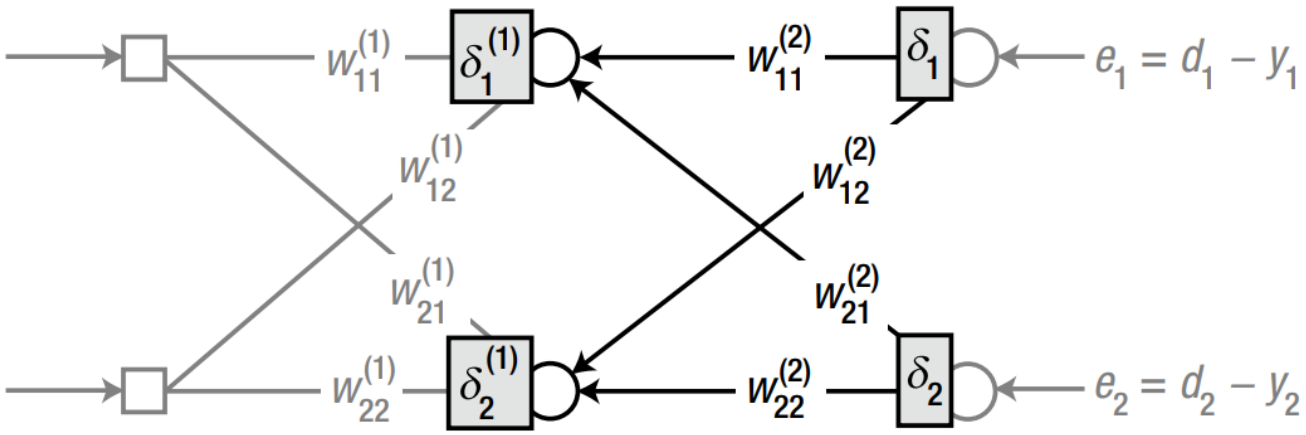
第一个输出节点：

$$e_1 = d_1 - y_1$$
$$\delta_1 = \varphi'(v_1)e_1$$

第二个输出节点：

$$e_2 = d_2 - y_2$$
$$\delta_2 = \varphi'(v_2)e_2$$

浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University



第一个输出节点：

$$e_1 = d_1 - y_1$$
$$\delta_1 = \varphi'(v_1)e_1$$

第二个输出节点：

$$e_2 = d_2 - y_2$$
$$\delta_2 = \varphi'(v_2)e_2$$

隐藏层第一个节点：

$$e_1^{(1)} = w_{11}^{(2)}\delta_1 + w_{21}^{(2)}\delta_2$$
$$\delta_1^{(1)} = \varphi'(v_1^{(1)})e_1^{(1)}$$

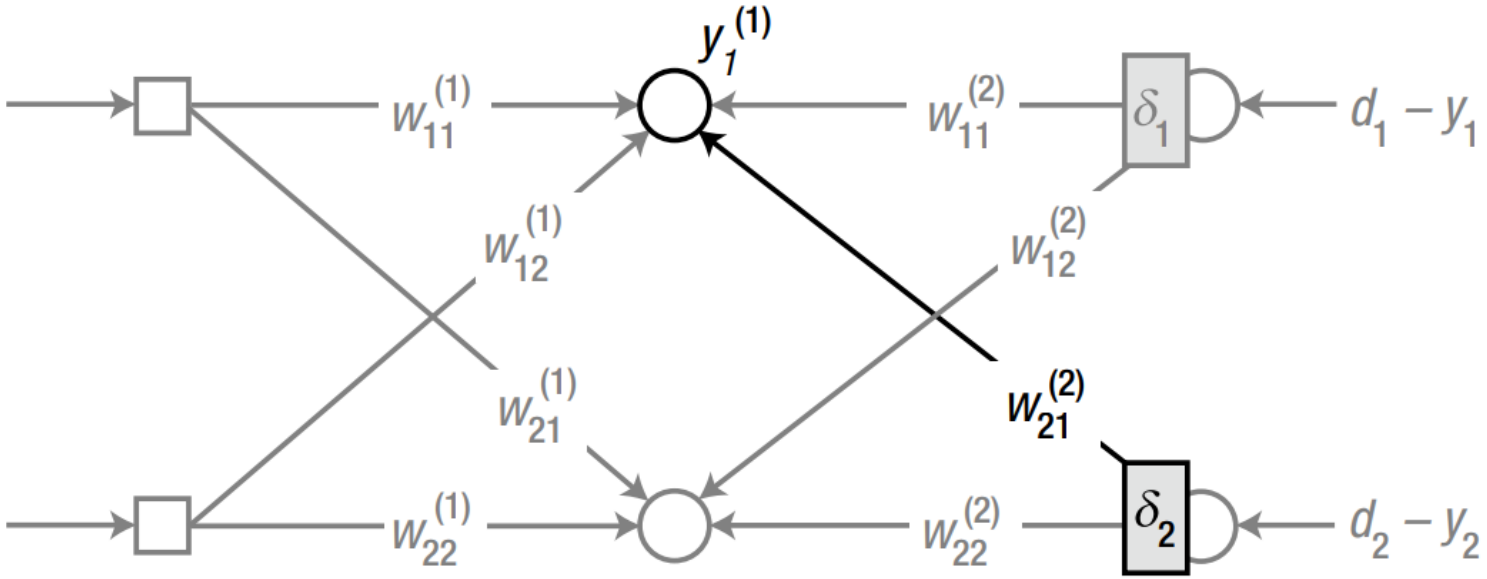
隐藏层第二个节点：

$$e_2^{(1)} = w_{12}^{(2)}\delta_1 + w_{22}^{(2)}\delta_2$$
$$\delta_2^{(1)} = \varphi'(v_2^{(1)})e_2^{(1)}$$

$$\begin{bmatrix} e_1^{(1)} \\ e_2^{(1)} \end{bmatrix} = \begin{bmatrix} w_{11}^{(2)} & w_{21}^{(2)} \\ w_{12}^{(2)} & w_{22}^{(2)} \end{bmatrix} \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix}$$

$$\begin{bmatrix} e_1^{(1)} \\ e_2^{(1)} \end{bmatrix} = W_2^T \begin{bmatrix} \delta_1 \\ \delta_2 \end{bmatrix}$$

误差调整:

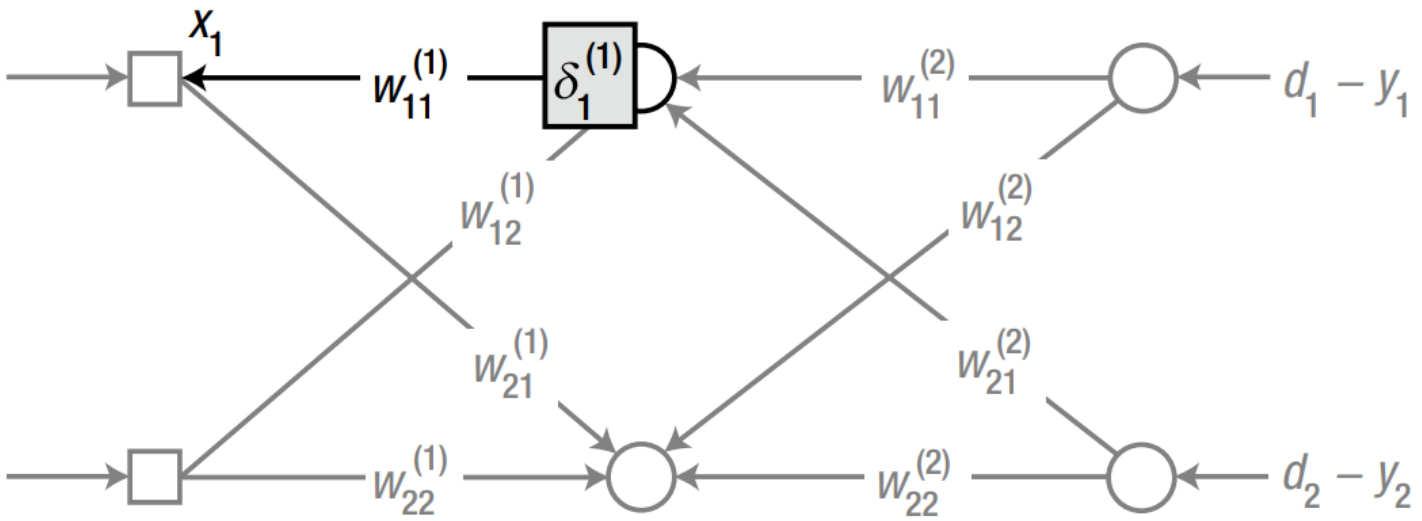


$$\Delta w_{ij} = \alpha \delta_i x_j$$

$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

$$w_{21}^{(2)} \leftarrow w_{21}^{(2)} + \alpha \delta_2 y_1^{(1)}$$

误差调整:



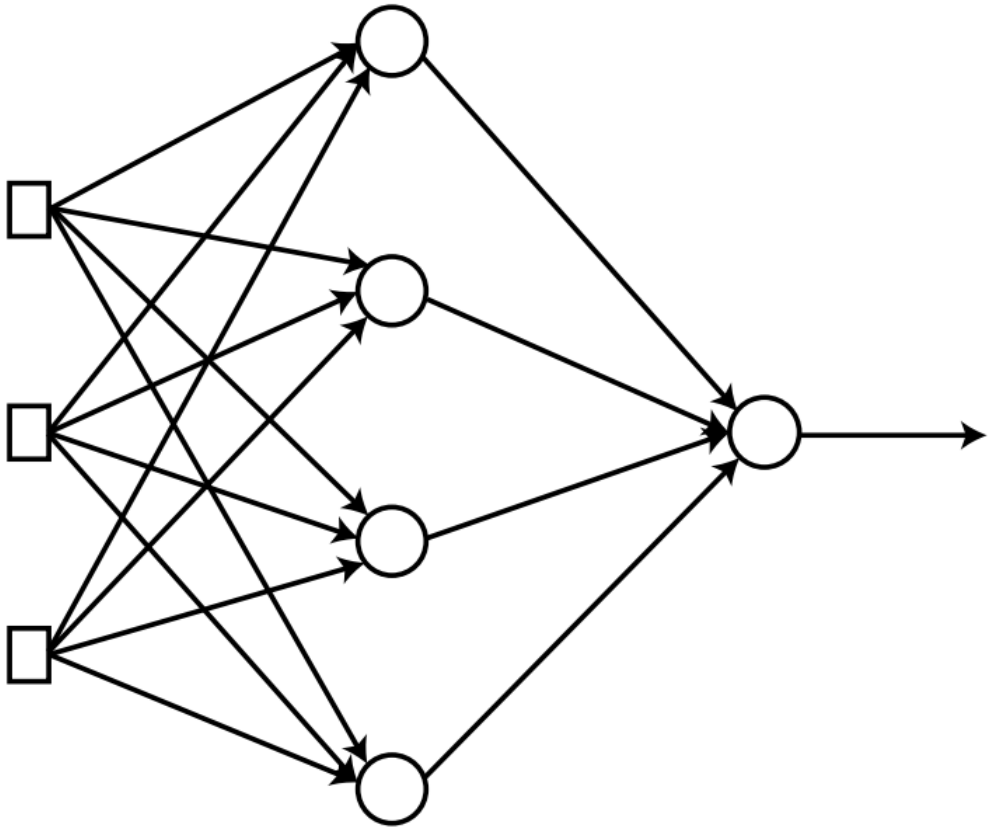
$$\Delta w_{ij} = \alpha \delta_i x_j$$
$$w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

$$w_{11}^{(1)} \leftarrow w_{11}^{(1)} + \alpha \delta_1^{(1)} x_1$$

浙江大学 信息与电子工程学院
College of Information Science & Electronic Engineering, Zhejiang University

1.2：实例BP训练

{ 0, 0, 1, **0** }
{ 0, 1, 1, **1** }
{ 1, 0, 1, **1** }
{ 1, 1, 1, **0** }



1.3 BP训练NN步骤

1. 初始化权重；
2. 通过训练数据对（输入、准确解），计算NN的输出、误差及Delta值

$$e = d - y \qquad \delta = \varphi'(v)e$$

3. 计算下一级的误差及Delta值

$$e^{(k)} = W^T\delta \qquad \delta^{(k)} = \varphi'\left(v^{(k)}\right)e^{(k)}$$

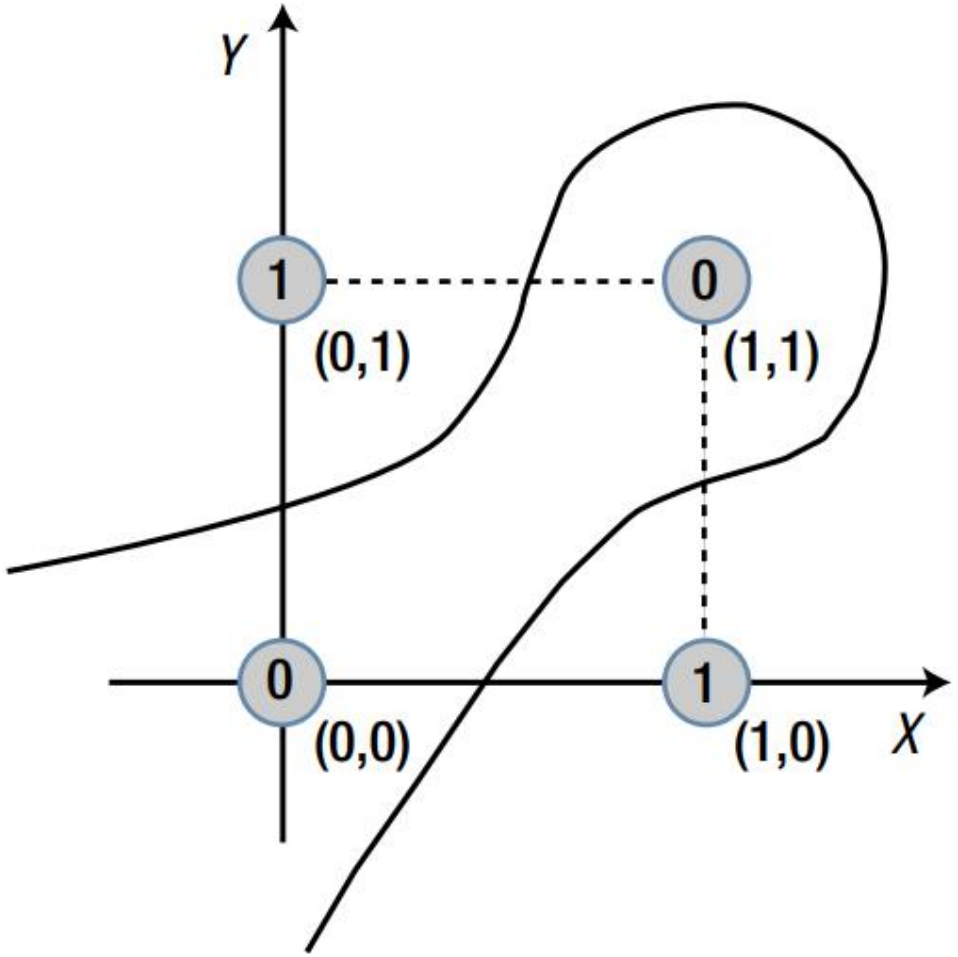
4. 重复第三步，直到所有层都传递完成
5. 根据下列公式调整权重

$$\Delta w_{ij} = \alpha\delta_i x_j \qquad w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$

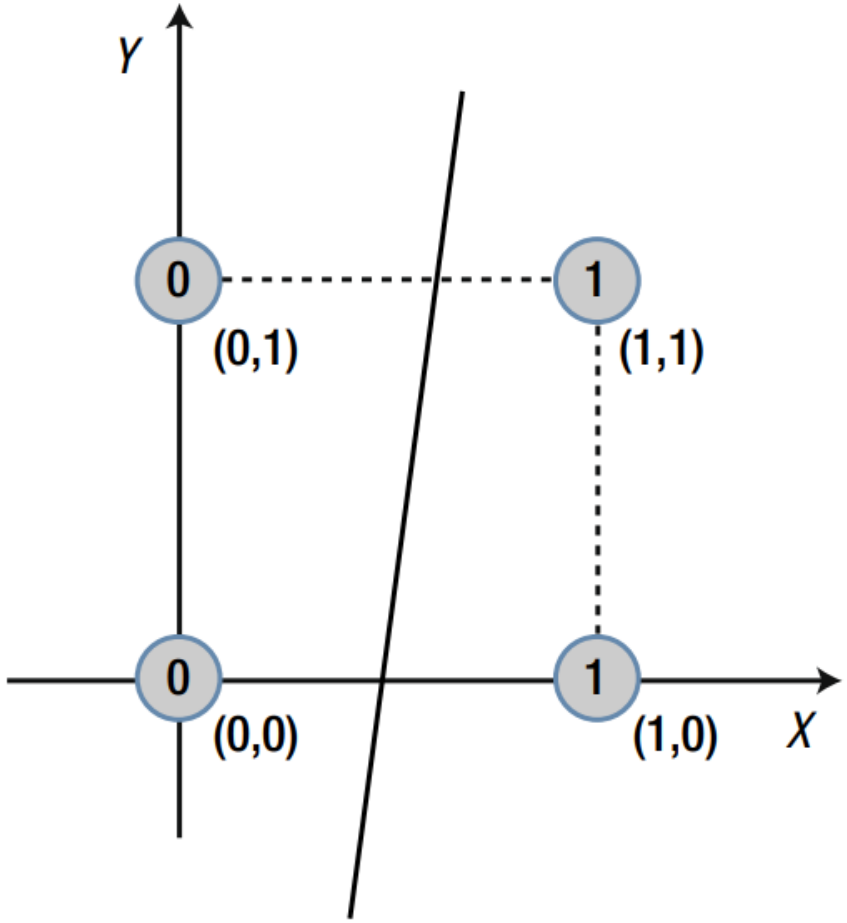
6. 重复2-5，直到遍历所有训练数据
7. 重复2-6直到网络达到训练要求；

实验5-2：

- 通过SGD训练方法、Sigmoid激活函数及BP规则，对上述神经网络进行训练，并输出训练后的结果。
- 利用实验5-1（上节课）的单层网络及SGD方法，用上述数据进行训练并测试，查看并对比两种网络输出结果的优劣，分析原因；



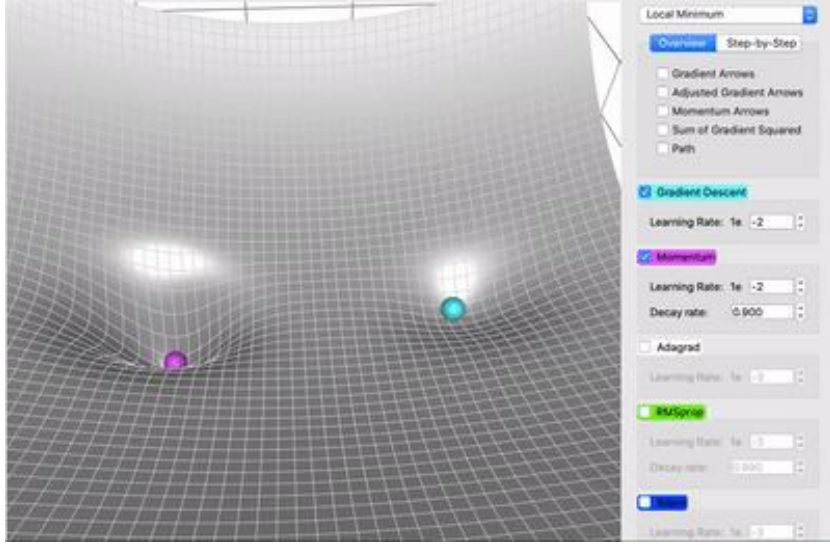
实验5-2



实验5-1

1.4：学习率、损失函数、 momentum

$$\Delta w_{ij} = \alpha \delta_i x_j \qquad w_{ij} \leftarrow w_{ij} + \Delta w_{ij}$$



Momentum：综合利用前面步骤的方向调整权重更新方向

$$\Delta w = \alpha \delta x$$

$$m = \Delta w + \beta m^{-}$$

$$w = w + m$$

$$m^{-} = m$$

$$m(0) = 0$$

$$m(1) = \Delta w(1) + \beta m(0) = \Delta w(1)$$

$$m(2) = \Delta w(2) + \beta m(1) = \Delta w(2) + \beta \Delta w(1)$$

$$m(3) = \Delta w(3) + \beta m(2) = \Delta w(3) + \beta \{\Delta w(2) + \beta \Delta w(1)\}$$

$$= \Delta w(3) + \beta \Delta w(2) + \beta^2 \Delta w(1)$$

$$\vdots$$

Momentum有利于逃脱局部最小，但是有些情况到达全局最小后仍然小范围振荡，需要额外的几步。

实验5-3：

- 将神经网络更新规则改为momentum，对上述神经网络进行训练，输出训练后的结果，比较不同β值momentum训练方法及不用momentum训练方法时，误差(真实结果与输出的MSE)随epoch变化趋势，并可视化结果。

Step 1:定义BP函数

```
def Backprop (W1, W2, X, D):  # D ground truth
    alpha = XXXX
    N = 4
    for k in range(N):
        x = XXXX
        d = XXXX
        v  = XXXX
        y  = XXXX

        dW1 = XXXX
        W1  = XXXX

        dW2 = XXXX
        W2  = XXXX
```

Step 1:定义BP-momentum函数

```
def BackPropMmt(W1, W2, X, D):
    alpha = XXXX
    beta  = XXXX
    mmt1 = np.zeros_like(W1)
    mmt2 = np.zeros_like(W2)

     XXXX
```